



# Computer Vision

## L05: Feature Detection and Matching (Lines and Shapes)

Prof. Dr. Mohammed Elmogy

Mansoura University  
Faculty of Computers & Information  
Dept. of Information Technology  
[elmogy@gmail.com](mailto:elmogy@gmail.com)

19. October 2025



# Outline

- Shape Features
- Successive Approximation
- Least Squares Fit
- RANSAC
- Line Hough Transform
- Circle Fitting
- Generalized Hough Transform
- Vanishing Points



# Outline

## Shape Features

Successive Approximation

Least Squares Fit

RANSAC

Line Hough Transform

Circle Fitting

Generalized Hough Transform

Vanishing Points



# Shape Features

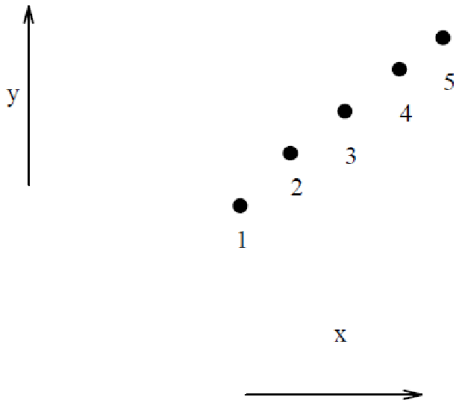
- ▶ Straight Lines.
- ▶ Circles and Ellipses.
- ▶ Arbitrary Shapes.



# Line Detection Applications

- ▶ Architectural modeling.
- ▶ Pose estimation in urban environments.
- ▶ Analysis of printed document layouts.

# How to Fit A Line?



$$y = mx + c$$



## How to Fit A Line? (cont'd)

- ▶ **Successive approximation**: **curve**  $\Rightarrow$  **piecewise-linear** **polyline** or a **B-spline** curve.
- ▶ **Least squares Fit** (**over constraint**)  $\Rightarrow$  more than two points are required.
- ▶ **RANSAC** (**constraint**)  $\Rightarrow$  only two points are required.
- ▶ **Hough Transform** (**under constraint**)  $\Rightarrow$  only one point is required.



# Outline

Shape Features

**Successive Approximation**

Least Squares Fit

RANSAC

Line Hough Transform

Circle Fitting

Generalized Hough Transform

Vanishing Points

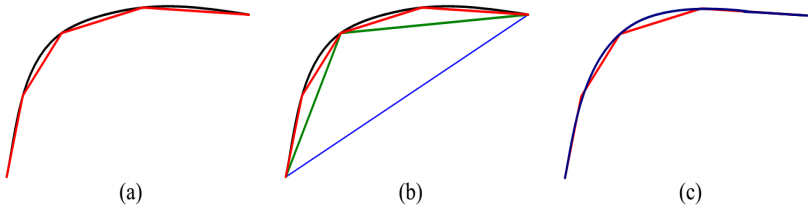




# Successive Approximation

- ▶ **Line simplification method** is used to approximate a curve with a **simpler representation**, e.g., as a **piecewise-linear polyline** or as a **B-spline curve**.
- ▶ One of the **oldest** and **simplest** methods is to **recursively subdivide the curve at the point furthest away** from the line joining the two endpoints.

## Approximating a Curve as a Polyline or B-Spline



1. Original curve and a polyline approximation shown in red.
2. Successive approximation by recursively finding points furthest away from the current approximation.
3. Smooth interpolating spline, shown in dark blue, fit to the polyline vertices.



# Outline

Shape Features

Successive Approximation

**Least Squares Fit**

RANSAC

Line Hough Transform

Circle Fitting

Generalized Hough Transform

Vanishing Points

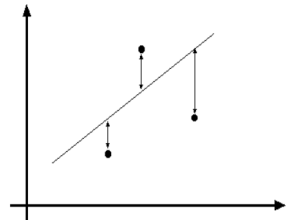
# Least Squares Fit

- Standard linear solution to estimating unknowns.
  - If we know which points belong to which line
  - Or if there is only one line

$$y = mx + c = f(x, m, c)$$

$$\text{Minimize } E = \sum_i [y_i - f(x_i, m, c)]^2$$

Take derivatives wrt  $m$  and  $c$  set them to 0



# Line Fitting

$$y = mx + c$$



$$y_1 = mx_1 + c$$

$$y_2 = mx_2 + c$$

$$\vdots$$

$$y_n = mx_n + c$$



$$\underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_B = \underbrace{\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & 1 \\ x_n & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} m \\ c \end{bmatrix}}_D \Rightarrow B = AD$$



$$A^T B = A^T A D$$

$$(A^T A)^{-1} A^T B = (A^T A)^{-1} (A^T A) D$$

$$D = (A^T A)^{-1} A^T B$$



# Outline

Shape Features

Successive Approximation

Least Squares Fit

**RANSAC**

Line Hough Transform

Circle Fitting

Generalized Hough Transform

Vanishing Points



# RANSAC (Random Sampling and Consensus)

1. Randomly select two points to fit a line.
2. Find the error between the estimated solution and all other points. If the error is less than tolerance, then quit, else go to step (1).

# Advantage and Disagvatage of RANSAC

## Advantage

No **accumulator array** is needed and so the algorithm can be more space efficient and potentially less prone to the choice of bin size.

## Disadvantage

Many **more hypotheses may need to be generated** and tested than those obtained by finding peaks in the accumulator array.



# Advantage and Disagvatage of RANSAC

## Advantage

No **accumulator array** is needed and so the algorithm can be more space efficient and potentially less prone to the choice of bin size.

## Disadvantage

Many **more hypotheses may need to be generated** and tested than those obtained by finding peaks in the accumulator array.



# Outline

Shape Features

Successive Approximation

Least Squares Fit

RANSAC

**Line Hough Transform**

Circle Fitting

Generalized Hough Transform

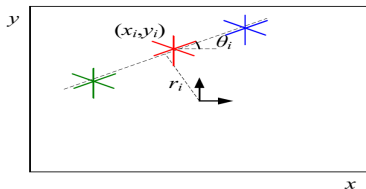
Vanishing Points



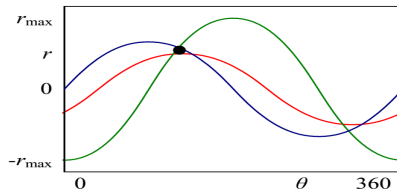
# Hough Transform

- ▶ It is a well-known technique for **having edges “vote” for plausible line locations.**
- ▶ Each edge point **votes for all possible lines passing through it,** and **lines corresponding to high accumulator or bin values are examined for potential line fits.**
- ▶ Unless the points on a line are truly punctate, a better approach is to use the local orientation information at each edgel to vote for a single accumulator cell.

# Original Hough Transform



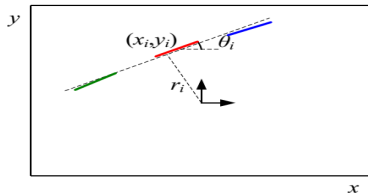
(a)



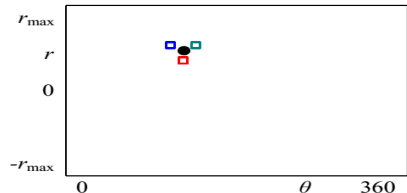
(b)

1. Each point votes for a complete family of potential lines  
 $r_i(\theta) = x_i \cos \theta + y_i \sin \theta$ .
2. Each pencil of lines sweeps out a sinusoid in  $(r, \theta)$ ; their intersection provides the desired line equation.

# Oriented Hough Transform



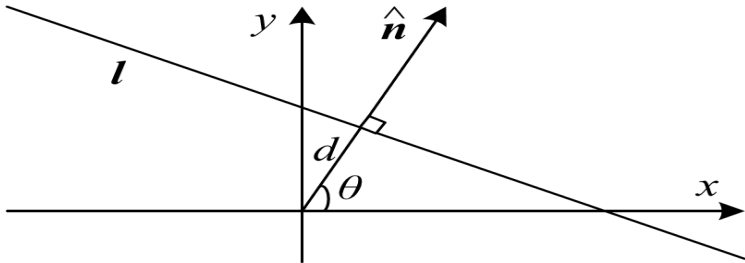
(a)



(b)

1. An edge re-parameterized in polar  $(r, \theta)$  coordinates, with  $\hat{n}_i = (\cos\theta_i, \sin\theta_i)$  and  $r_i = \hat{n}_i \cdot x_i$ .
2.  $(r, \theta)$  accumulator array, showing the votes for the three edges marked in red, green, and blue.

2D line equation expressed in terms of the normal  $\hat{n}$  and distance to the origin  $d$



# Outline of a Hough Transform Algorithm Based on Oriented Edge Segments

**procedure** *Hough*( $\{(x, y, \theta)\}$ ):

1. Clear the accumulator array.
2. For each detected edgel at location  $(x, y)$  and orientation  $\theta = \tan^{-1} n_y/n_x$ , compute the value of

$$d = x n_x + y n_y$$

and increment the accumulator corresponding to  $(\theta, d)$ .

3. Find the peaks in the accumulator corresponding to lines.
4. Optionally re-fit the lines to the constituent edgels.

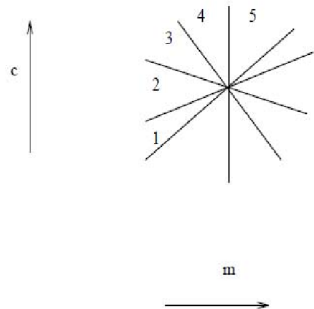
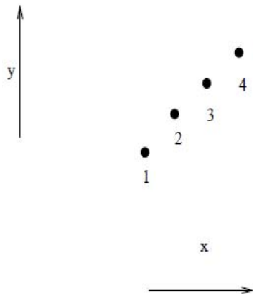


## Line Fitting: Hough Transform

- ▶ Line equation:  $y = mx + c$ , where  $m$  is slope and  $c$  is y-intercept.
- ▶ Rewrite this equation:  $c = (-x)m + y$ .
- ▶ For particular edge point  $i$  this becomes:  $c = (-x_i)m + y_i$ .
- ▶ This is an equation of a line in  $(c,m)$  space.



## Line Fitting: Hough Transform (cont'd)



$$c = (-x_i)m + y_i$$

# Hough Transform Algorithm for Fitting Straight Lines

1. Quantize the parameter space  
 $P[c_{min}, \dots, c_{max}, m_{min}, \dots, m_{max}]$ .
2. For each edge point  $(x, y)$  do  
for  $(m = m_{min}, m \leq m_{max}, m++)$  do  
 $c = (-x)m + y$ ,  
 $P[c, m] = P[c, m] + 1$ .
3. Find the local maxima in the parameter space.

# Polar Form of Equation of Line

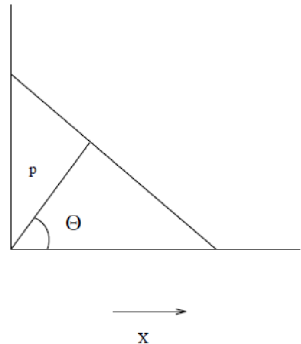
$$c_i = (-x)m_j + y$$

Problematic for vertical lines  
 $m$  and  $c$  grow to infinity



$$p = x \cos \theta + y \sin \theta$$

Use  $\theta$  from gradient

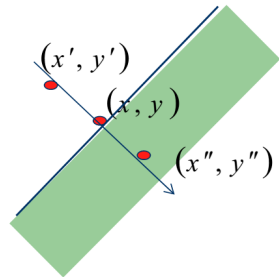


# Image Gradient

$(S_x, S_y)$  Gradient Vector

$$\text{magnitude} = \sqrt{(S_x^2 + S_y^2)}$$

$$\text{direction} = \theta = \tan^{-1} \frac{S_y}{S_x}$$



# Hough Transform for Polar Form of Equation of Line

1. Quantize the parameter space

$$P[\theta_{min}, \dots, \theta_{max}, p_{min}, \dots, p_{max}].$$

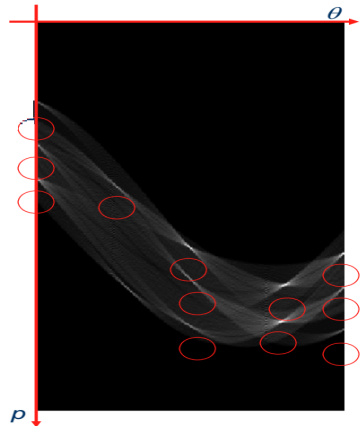
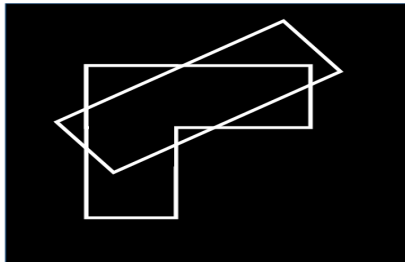
2. For each edge point  $(x, y)$  do

$$p = x \cos \theta + y \sin \theta,$$

$$P[\theta, p] = P[\theta, p] + 1.$$

3. Find the local maxima in the parameter space.

# Line Fitting





## Difficulties

- ▶ What is the increments for  $\theta$  and  $p$ .
  - ▶ Too large? We cannot distinguish between different lines.
  - ▶ Too small? noise causes lines to be missed.

# General Note

## Application Dependant

There is **no clear consensus** on which line estimation technique performs best. It is therefore a good idea to **think carefully about the problem** at hand and to implement several approaches (**successive approximation**, **Hough**, and **RANSAC**) to determine the one that works best for your application.





# Outline

Shape Features

Successive Approximation

Least Squares Fit

RANSAC

Line Hough Transform

**Circle Fitting**

Generalized Hough Transform

Vanishing Points

# Circle Fitting

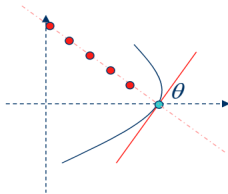
- Similar to line fitting
  - Three unknowns

$$(x - x_o)^2 + (y - y_o)^2 - r^2 = 0$$

- Construct a 3D accumulator array **A**
  - Dimensions:  $x_0, y_0, r$
- Fix one of the parameters and loop for the others
- Increment corresponding entry in **A**.
- Find the local maxima in **A**

## More Practical Circle Fitting

- Use the tangent direction  $\theta$  at the edge point



- Compute  $x_0, y_0$  given  $x, y, r$

$$x_0 = x - r \cos \theta$$

$$y_0 = y - r \sin \theta$$

# Circite Hough Transform Algorithm

1. Quantize the parameter space

$$P[x_{0min}, \dots, x_{0max}, y_{0min}, \dots, y_{0max}, r_{min}, \dots, r_{max}].$$

2. For each edge point  $(x, y)$  do  
For  $(r = r_{min}, r \leq r_{max}, r++)$

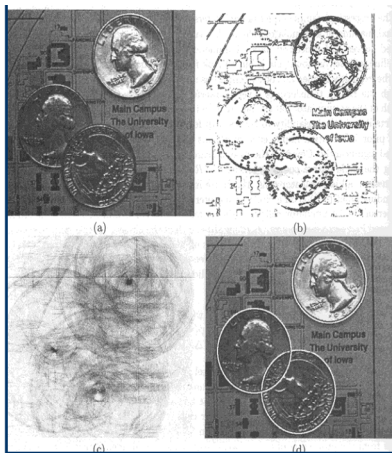
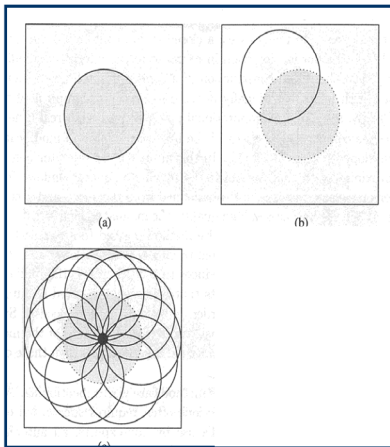
$$x_0 = x - r \cos \theta,$$

$$y_0 = y - r \sin \theta,$$

$$P[x_0, y_0, r] = P[x_0, y_0, r] + 1.$$

3. Find the local maxima in the parameter space.

# Examples





# Outline

Shape Features

Successive Approximation

Least Squares Fit

RANSAC

Line Hough Transform

Circle Fitting

**Generalized Hough Transform**

Vanishing Points

# Generalized Hough Transform

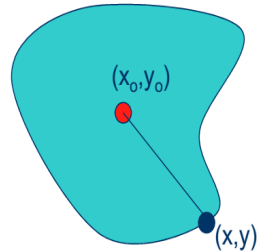
- ▶ Used for shapes with no analytical expression.
- ▶ Requires training:
  - ▶ Object of known shape
  - ▶ Generate model (R-table)
- ▶ Similar approach to line and circle fitting during detection.

## Generating R-table

- Compute centroid
- For each edge compute its distance to centroid

$$r = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x - x_0 \\ y - y_0 \end{bmatrix}$$

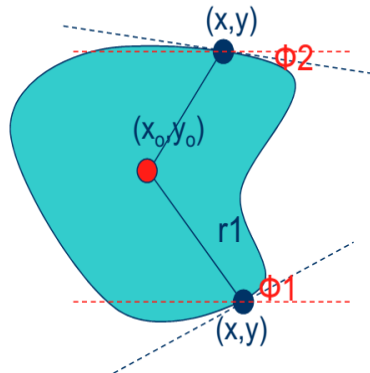
- Find edge orientation (gradient angle)
- Construct a table of angles and  $r$  values





## Generating R-table (cont'd)

|          |                       |
|----------|-----------------------|
| $\phi 1$ | $r1, r2, r3 \dots$    |
| $\phi 2$ | $r14, r21, r23 \dots$ |
| $\phi 3$ | $r41, r42, r33 \dots$ |
| $\phi 4$ | $r10, r12, r13 \dots$ |



## Detecting shape

- known
  - Edge points  $(x,y)$
  - Gradient angle at every edge point  $\theta$
  - R-table of the shape needs to be determined
- For each edge point find  $\theta$  store it in corresponding row of R-table
- Create an accumulator array of 2D  $(x,y)$

# Generalized Hough Transform

1. Quantize the parameter space  $P[x_{cmin}, \dots, x_{cmax}, y_{cmin}, \dots, y_{cmax}]$ .
2. For each edge point  $(x, y)$  do  
compute  $\phi(x, y)$   
for each table entry for  $\phi$  do

$$x_c = x + x'$$

$$y_c = y + y'$$

$$P[x_c, y_c] = P[x_c, y_c] + 1.$$

3. Find the local maxima in the parameter space.

# Rotation and Scale Invariance

- **Rotation around Z-axis**

$$x' = x \cos \alpha - y \sin \alpha$$

$$y' = x \sin \alpha + y \cos \alpha$$

- **Scaling**

$$x' = sx$$

$$y' = sy$$

- **Rotation+scaling**

$$x' = s(x \cos \alpha - y \sin \alpha)$$

$$y' = s(x \sin \alpha + y \cos \alpha)$$



# Outline

Shape Features

Successive Approximation

Least Squares Fit

RANSAC

Line Hough Transform

Circle Fitting

Generalized Hough Transform

**Vanishing Points**



# Vanishing Points

- ▶ Structurally important lines have the **same vanishing point** because they are **parallel in 3D**.
- ▶ Examples of such lines are:
  - ▶ Horizontal and vertical building edges.
  - ▶ Zebra crossings.
  - ▶ Railway tracks.
  - ▶ The edges of furniture such as tables and dressers.
  - ▶ Ubiquitous calibration pattern.



# Importance of Vanishing Points

- ▶ It can help refine the position of lines in the image.
- ▶ It help determine the intrinsic and extrinsic orientation of the camera.



## References

1. Richard Szeliski, Computer Vision: Algorithms and Applications, 2nd Ed., Springer-Verlag London, 2020.
2. Mubarak Shah CV Course: **Lecture 17**  
(<https://www.youtube.com/watch?v=tAGiK1Ev6Ng&list=PLd3hISJs9&index=17>)
3. David Forsyth and Jean Ponce: Computer Vision: A Modern Approach, 2nd Ed., Pearson Education, Inc., 2012.